

Projektöversikt — Gemenskap

Contents

Projektöversikt — Gemenskap (uppdaterad)	1
1. Vad är det här?	1
2. Mappstruktur	1
3. Backend (server/)	2
4. Frontend (public/)	4
5. Datalager och kommunikation	4
6. AI och ML — hur det faktiskt funkar	4
7. Skillnader mot den gamla teknisk-rapport.md	7
8. Vad jag skulle ifrågasätta (kort)	8

Projektöversikt — Gemenskap (uppdaterad)

Det här dokumentet beskriver hur projektet faktiskt fungerar idag, baserat på koden i mappen. Det ersätter den utdaterade teknisk-rapport.md (som beskriver en äldre version utan moderation, SSE, AI-skapade grupper och cache av ML-modellen).

1. Vad är det här?

Gemenskap är en lokal, helt självkörande webbprototyp av en community-plattform för äldre. Frontend är ren HTML/CSS/JS utan ramverk. Backend är en Express-server med SQLite (better-sqlite3) som körs i samma process. ML-modellen för matchning tränas eller laddas vid serverstart, allt lokalt. Inga externa tjänster — SMS-koden är simulerad och visas både i terminalen och i en gul dev-banner i webbläsaren.

Starta med `npm install`, `npm run seed`, `npm run dev`. Sedan ligger sajten på `http://localhost:3000`.

2. Mappstruktur

```
prototyp v3/
├── server/          Backend (Express + SQLite)
├── public/          Frontend (HTML/CSS/JS, statiska filer)
├── data/            dataset.json (seed-data), app.db (skapas av seed), model.json (ML-
cache)
├── scripts/         Engångsskript som bygger/berikar dataset.json
├── docs/            Gammal teknisk rapport (kopia)
└── tools/           Inbäddad Node.js för Windows
```

```
|— node_modules/  npm-paket
|— package.json
|— README.md
```

3. Backend (server/)

Servern är uppdelad i en handfull moduler som alla körs i samma Node-process.

`server.js` är ingången. Den startar Express, slår på `cookieParser` och `JSON-body-parsing` (max 5 MB), monterar API-routerna under `/api/*`, och serverar `public/` som statiska filer. Vid uppstart triggas också `ml.loadAndTrain()` så rekommendationsmotor är redo när första requesten kommer in. En generisk error-handler i botten ger lokaliserade 500-svar (svenska + engelska) i samma format som övriga fel.

`db.js` initierar SQLite-databasen i `data/app.db` med `WAL`-läge och `foreign_keys = ON`. Den definierar hela schemat och kör några idempotenta migrationer: lägger till `attempts`-kolumnen på `auth_codes` (brute-force-skydd), lägger till `creator_id` på `communities` (för att kunna spåra vem som skapat en grupp), samt rensar redan tomma `communities` vid uppstart. Tabellerna är: `users`, `user_interests`, `communities`, `community_interests`, `memberships`, `posts`, `events`, `event_rsvps`, `messages`, `auth_codes`, `user_blocks`, `reports`, `sessions`. Maxgränsen `MAX_MEMBERS = 40` per community är hårdkodad här.

`auth.js` hanterar SMS-baserad inloggning. `POST /api/auth/request-code` genererar en 6-siffrig kod (giltig 10 min), `rate-limited` till 5 koder per nummer per timme, och loggar koden till konsolen + skickar tillbaka den i `dev_code`-fältet. `POST /api/auth/verify` jämför koden med `crypto.timingSafeEqual` för att undvika timing-läckor, räknar upp `attempts` vid fel och invaliderar koden efter fem felaktiga försök. När koden stämmer skapas/hämtas användaren, gamla sessioner rensas (>90 dagar) och totalantalet kapas vid 10 sessioner per användare. En 64-tecken hex-token sparas i `sessions` och sätts som `httpOnly/sameSite=lax-cookie` (secure i produktion). `GET /api/auth/me` returnerar inloggad användare med intressen, medlemskap och `createdCommunities` (även de man lämnat — så man kan hitta tillbaka). `requireAuth`-middleware exporteras och används av nästan alla andra routrar.

`users.js` hanterar profilen: `PUT /api/users/me` uppdaterar `namn/stad/bio/språk/intressen`, `POST /api/users/me/avatar` tar emot en data-URL och validerar att bufferten faktiskt börjar med `JPEG/PNG/WebP-magic-bytes` (Content-Type-headern går att fejka, magic bytes svårare), max 5 MB. `GET /api/users/me/export` ger en GDPR-export (artikel 20) som nedladdningsbar JSON med all användardata. `DELETE /api/users/me` raderar allt (GDPR). `GET /api/users/:id` är skyddad så att man bara ser andra profiler om man delar minst ett community — annars 403. Förhindrar IDOR-scrapning genom att iterera `/api/users/1, /2, /3`.

`communities.js` listar (`GET /`), föreslår (`GET /suggest` — anropar `matching/ML`), tillhandahåller AI-namnpreview (`POST /preview` — kallar `community-namer.js`, `rate-limited` till 10/min), skapar nytt community (`POST /` med validering, max 3 nya per dygn, transaktion som skriver community + intressen + creator-medlemskap atomärt), visar detaljer (`GET /:id` — inkluderar `isMember` och `isCreator`-flaggor), listar medlemmar, samt `join/leave`. `Leave`-routen körs som en transaktion och raderar gruppen om sista medlemmen lämnar (`CASCADE` rensar `inlägg`, `event`, etc.).

`posts.js` är monterad under `/api/communities/:communityId/posts`. Hämtar de 100

senaste inläggen i en grupp, men filtrerar bort inlägg från författare som blockerats åt något håll (en blockerad person ska inte se eller bli sedd). Skapar nytt inlägg (rate limit 10/min per användare och community, max 4000 tecken, kräver medlemskap).

events.js listar kommande event (/upcoming för dina grupper, /community/:id för en specifik grupp, inklusive event från senaste 24h så att precis avslutade event inte försvinner direkt), skapar event (kräver medlemskap, validerar längder, kräver framtida datum max 5 år fram), samt RSVP via upsert (going/maybe/no).

messages.js är där det blivit störst skillnad mot den gamla rapporten. Polling är borttaget — istället används **Server-Sent Events**. Varje öppen flik ansluter till GET /api/messages/stream som håller en hängande HTTP-respons. När ett nytt meddelande sparas pushar sendEvent() det till alla aktiva strömmar för avsändare och mottagare. En heartbeat-kommentar var 25:e sekund håller anslutningen levande. När en mottagare öppnar en tråd skickas också ett read-event tillbaka till avsändaren, så lästkvitton kommer i realtid. Blockering respekteras både vid hämtning av trådar och vid skickande. Max 20 meddelanden per minut till samma mottagare, max 2000 tecken per meddelande.

moderation.js är en helt ny modul som inte fanns i gamla rapporten. Hanterar POST/DELETE /api/moderation/blocks/:userId (blockera/avblockera) och GET /api/moderation/blocks/:userId (lista mina blockerade). POST /api/moderation/reports rapporterar inlägg, användare eller meddelanden (rate limit 5/min, dubbletskydd inom 24h, sparas i reports-tabellen även om ingen agerar — kanalen finns och datan fångas).

ml.js är rekommendationsmotorn, byggd från grunden utan ML-bibliotek. Den kombinerar fyra signaler i en ensemble: **Jaccard-likhet på intressen (0.45)**, **cosine similarity på feature-vektor (0.25)**, **neuralt nätverk (0.15)** och **collaborative filtering (0.15)**. Om användaren angett intressen men ett community har noll gemensamma multipliceras poängen med 0.25 — geografi får inte trycka upp irrelevanta grupper. Det neurala nätverket är ett feedforward-nät $41 \rightarrow 32$ (ReLU) $\rightarrow 16$ (ReLU) $\rightarrow N$ communities (sigmoid), tränat med SGD 80 epoker på medlemskapsmönster i dataset.json. Vikterna cachas till data/model.json och laddas in om datasetet inte ändrats sen sist — så omstarter går snabbt. Feature-vektorn är 41-dimensionell (32 fördefinierade intressen + 9 städer one-hot). matching.js använder motorn och lägger på en mjuk **geografisk multiplikator** (samma stad $\times 1.00$, <80 km $\times 0.90$, <200 km $\times 0.75$, ≥ 200 km $\times 0.55$) baserad på Haversine-avstånd mellan 24 svenska städer. En regelbaserad fallback finns om ML inte är redo.

community-namer.js genererar deterministiskt namn + beskrivning för en ny grupp baserat på intressen och stad. Inget LLM-anrop — bara en hash + mall per kategori (outdoor, sport, create, culture, food, health, mixed). Bra trade-off för en demo: gratis, snabbt, transparent.

rate-limit.js är en enkel in-memory sliding-window rate-limiter som factory-funktion. Används av meddelanden, inlägg, rapporter och AI-preview. Periodisk sweep var 5:e minut rensar gamla nycklar.

seed.js läser data/dataset.json och fyller på databasen i transaktioner. Datasetet har genererats av scripts/build-dataset-from-meetup.js (från riktig Meetup-data, KDD 2012) och utökats av scripts/enrich-dataset.js.

4. Frontend (public/)

Nio HTML-sidor — index, login, onboarding, home, community, events, messages, profile, browse — varje sida laddas med vanlig fullständig sidnavigering, inga client-side routes. Varje sida laddar samma kärn-skript i bestämd ordning (i18n → api → auth → components → ev. interests/tutorial → sidspecifikt skript i js/page/).

Modulmönstret är IIFE: varje fil definierar sina privata funktioner i en sluten scope och exponerar bara ett objekt på window. window.API är en fetch-wrapper som kastar ett fel med userMessage på det aktuella språket. window.Auth cachar inloggad användare och har requireUser()/requireOnboardedUser() som redirectar vid behov. window.I18n håller alla översättningar i ett inbäddat objekt och växlar språk via ett langchange-event på document. window.UI har de delade UI-byggstenarna (header, bottennav, alert, avatar, dev-banner). window.Interests definierar intressekategorierna och renderar chip-väljaren. window.Tutorial är overlay-tutorialen för förstagångsanvändare.

Det finns några nyare skript som inte fanns i gamla rapporten: font-size.js (textstorleksväxlare), tips.js (säkerhets/scam-tips på olika sidor), user-storage.js (prefixar localStorage-nycklar med användar-ID så två användare på samma enhet inte trampar på varandras tutorial-flagga eller "senaste community").

page/messages.js använder EventSource('/api/messages/stream') för att ta emot meddelanden i realtid och har en automatisk reconnect efter 3s om strömmen stängs. Den har också en **PII-detektor** som varnar om användaren råkar skriva personnummer, IBAN, kreditkortsnummer eller mobilnummer i ett meddelande — ett konkret skydd mot scams och romansbedrägerier som är extra relevanta för målgruppen.

CSS:en (styles.css, ~1000 rader) bygger på CSS-variabler: 20 px basfont, 58 px knapphöjd, varm skandinavisk palett (salviagrön, sienna, honung), tydlig orange fokusring och prefers-reduced-motion-stöd. Atkinson Hyperlegible (font designat för nedsatt syn) laddas från Google Fonts.

5. Datalager och kommunikation

All kommunikation går över HTTP/JSON mot localhost:3000. Sessionscooken sid (httpOnly, 1 år) är det enda serverhanterade tillståndet. Klient-state delas i tre kategorier: persistent i localStorage (språk, tutorial, senaste community — prefixade per användare), in-memory i modulvariabler per sida, och URL-parametrar (?id=, ?with=).

Felsvar har alltid samma form — { error, message_sv, message_en } — och frontenden plockar rätt språk via I18n.getLang(). Alla felmeddelanden är skrivna lösningsorienterat ("Vi hittar ingen kod för det här numret. Tryck på Skicka ny kod nedan.").

6. AI och ML — hur det faktiskt funkar

Det finns tre saker i projektet som etiketteras eller känns som "AI" i gränssnittet. Två av dem är riktig ML, en är det inte alls. Värt att hålla isär.

6.1 Communityrekommendationen (riktig ML)

Det här är hjärtat i "AI-matchning" som visas under onboarding och på hemsidan. Den ligger i server/ml.js och anropas från server/matching.js. Allt är skrivet från grunden

— inga externa ML-bibliotek (ingen TensorFlow, ingen scikit-learn). Det betyder att hela logiken går att läsa och felsöka i ren JavaScript, vilket är ovanligt och pedagogiskt smart för en demo.

Steg 1 — Feature-vektorer. Varje användare och varje community kodas som en 41-dimensionell binär vektor: 32 fördefinierade intressen + 9 städer, alla one-hot. Om du gillar Schack och bor i Göteborg blir alla positioner noll utom positionen för "Schack" och positionen för "Göteborg", som blir 1. Communities kodas exakt likadant utifrån sina taggar och sin stad. Eftersom användare och community lever i samma vektorrum kan vi mäta likhet mellan dem direkt.

Steg 2 — Fyra parallella signaler. För varje community beräknas fyra olika "scores" som mäter passform på olika sätt:

- **Jaccard-likhet** mäter ren intresseöverlapp: antal gemensamma intressen delat med unionen. Om du har 4 intressen, communityt har 5, och 3 är gemensamma $\rightarrow \text{Jaccard} = 3 / (4+5-3) = 0,5$. Helt blind för stad. Det är den starkaste signalen i ensemblen (45 %) eftersom den är det mest direkta beviset på att ni passar ihop.
- **Cosine similarity** är vinkeln mellan de två 41-dimensionella vektorerna (egentligen $\text{dot}(a,b) / (||a|| \cdot ||b||)$). Skillnaden mot Jaccard är att denna räknar med staden också — två stockholmare med liknande intressen får högre cosine än en stockholmare och en malmöbo med samma intressen.
- **Neuralt nätverk** ger en lärd sannolikhet att just *du* skulle gå med i just det här communityt baserat på mönster i hela datasetet (mer i 6.2).
- **Collaborative filtering** är det klassiska "användare som är lika dig gick också med i X". Tekniskt: hitta de 15 mest lika användarna (cosine på feature-vektor), summera vilka communities de är medlemmar i (viktat med likhet), normalisera till 0-1.

Steg 3 — Ensemble. De fyra signalerna vägs ihop i `recommend()`:

```
combined = jaccard*0.45 + cosine*0.25 + nn*0.15 + collab*0.15
```

Vikterna är hårdkodade och inte lärda. Det är ett medvetet val — Jaccard är direkt evidens, det neurala nätverket och CF är mer indirekta. Om någon har angett intressen men ett community har **noll** gemensamma intressen multipliceras totalpoängen med 0,25. Det är straffet som hindrar ett geografiskt nära men irrelevant community från att bubbla upp på de andra signalerna.

Steg 4 — Geografisk multiplikator. Efter ensemble-poängen lägger `matching.js` på en mjuk geografi-faktor baserad på Haversine-avstånd mellan 24 svenska städer:

Avstånd	Faktor
Samma stad	1,00
< 80 km	0,90
< 200 km	0,75
≥ 200 km	0,55

Notera ordningen: matchning först, geografi sen. En perfekt schackmatchning i Lund tappar ~10 % poäng för en användare i Göteborg, men slår fortfarande en svag matchning i Göteborg. Det är rätt prioritering för en intressedriven sajt.

6.2 Det neurala nätverket

Det är ett vanilla feedforward-nät, byggt med Float64Arrays och for-loopar:

Input(41) → Dense(32, ReLU) → Dense(16, ReLU) → Dense(N, Sigmoid)

Där N är antalet communities i datasetet. Output är alltså en vektor där varje element är "sannolikheten att den här användaren skulle gå med i community i". Sigmoid (inte softmax) är rätt val här eftersom medlemskap är **multilabel** — du kan vara i flera communities samtidigt, det är inte en mutually exclusive klassificering.

Träningen är textbook:

- **Initialisering:** Xavier (scale = $\sqrt{2/(in+out)}$)).
- **Forward pass:** matrismultiplikation + bias, ReLU på dolda lager, sigmoid på output.
- **Loss:** binary cross-entropy per output-nod, summerad.
- **Backprop:** standard SGD, derivata av BCE+sigmoid förenklas till (output - target), ReLU-derivatan är 1 om aktivering > 0.
- **Hyperparametrar:** 80 epoker, learning rate 0,005, en sample i taget (ingen batching).
- **Träningsdata:** för varje användare är input = feature-vektor, target = vektor med 1:or där användaren är medlem och 0:or annars.

Allt körs i Node-tråden vid serverstart. På 300 användare och ~57 communities tar det under en sekund. Vid första körningen sparas vikterna till data/model.json. Vid nästa start jämförs mtime mellan dataset.json och model.json — har datan inte ändrats laddas vikterna direkt och träningen hoppas över.

Vad nätverket faktiskt lär sig: mönster i medlemskapen som går utöver direkt intresseöverlapp. Till exempel: om alla med Schack + Bridge i Göteborg också är med i ett visst Boule-community, så lär nätverket sig att rekommendera det Boule-communityt även till nya schack/bridge-spelare som inte explicit angett Boule. Det är värdet jämfört med Jaccard.

6.3 Community-namer (inte AI)

server/community-namer.js etiketteras som "AI-förslag" i UI:t när någon skapar en grupp. Det är inte ML och inget LLM-anrop — det är ren heuristik. Den hittar dominerande kategori bland intressena (outdoor, sport, create, culture, food, health eller mixed), hashar inputen deterministiskt med FNV-1a och väljer en mall ur en lista per kategori. Sedan klistras stadsnamnet på.

Det är ett medvetet val (gratis, snabbt, transparent, ingen extern beroende) och funkar för en demo. Men "AI" i UI:t är överdrivet — om en användare skulle granska skulle de upptäcka att samma intressen + stad alltid ger exakt samma namn. Antingen ändra etiketten till "föreslaget namn" eller seeda hashen med något mer än bara intressen + stad.

6.4 PII-detektor (inte AI)

page/messages.js har en uppsättning regex-mönster (PII_PATTERNS) som varnar om någon skriver personnummer, IBAN, kreditkort eller mobilnummer. Det är ren mönster-

matchning, men funktionellt sett en av de mer värdefulla “smarta” sakerna i appen för målgruppen — det är just sådant scammers ber äldre om över chatt.

6.5 Var ML-resultatet visas

I onboarding och på hemsidan ser användaren tre kort med kommentarer som “matchar dina intressen”, “i din stad” och liknande. Bakom kulisserna får frontenden tillbaka mer än bara score — `mlDetail` innehåller `nnScore`, `cosineSim`, `collabScore` och `geoFactor` som procent. Det används i UI:t (om man kollar) för att visa *varför* ett förslag dök upp. Det är ovanligt transparent jämfört med vanliga rekommendationssystem och en bra sak att behålla.

6.6 Ärlig värdering av ML-delen

Mentorsblick på det här:

- **Det neurala nätverket gör mindre nytta än man tror.** Det väger 15 %, det tränas på exakt samma feature-vektor som cosine-likheten redan använder, och det lär sig från ett seed-dataset där medlemskap till stor del är genererat utifrån just intresseöverlapp. Risken är att NN bara approximerar Jaccard på ett dyrare sätt. Värdet realiseras först när du har riktiga, ojämna medlemskapsmönster (t.ex. när någon med Schack också ofta råkar gilla Bridge utan att ha angett det).
- **Ensemble-vikterna är gissade, inte mätta.** Det finns ingen utvärdering någons i koden — ingen train/test-split, ingen precision/recall, inget A/B-mått. För en demo är det rimligt; om du vill förbättra matchningen är det första steget att skapa en utvärderingsrutin (även enkel: “håll ut 10 % av medlemskapen, predicera, mät hur ofta sant medlemskap är top-k”).
- **Cold start är OK men inte bra.** En helt ny användare utan intressen får mest geografi + collaborative filtering (som inte vet något om dem). Att noll-överlappstraffet bara aktiveras när användaren *har* intressen är klokt — det räddar cold start från att straffas hårt.
- **Inget online-lärande.** Modellen vet inget om vad användare faktiskt klickar på eller går med i efter rekommendationen. Den tränas en gång på seed-datat och sedan aldrig om. För en prototyp helt OK, men för verklig drift skulle nästa steg vara att periodiskt träna om mot uppdaterade memberships.
- **from-scratch-implementeringen är pedagogisk fin men begränsande.** Du har offrat snabbhet (vektorerade ops) och funktionalitet (regularisering, dropout, momentum, mini-batches) för att slippa beroenden. Helt rätt val för en demo som ska vara läsbar; fel val om datasetet växer mot tusentals användare och hundratals communities.

Sammantaget är ML-delen mer ärligt beskriven som “ett genomtänkt ensemble-rekommendationssystem med ett neuralt nätverk som en av komponenterna” än som “AI-driven matchning”. Det är inte sämre — det är faktiskt mer användbart än ett nätverk skulle vara ensamt — men det är inte magin som ordet “AI” antyder.

7. Skillnader mot den gamla teknisk-rapport.md

Den gamla rapporten är inte fel, men den missar flera saker som finns i koden idag:

- **Realtid:** SSE har ersatt 5-sekunders polling för meddelanden, både push av nya meddelanden och realtidskvitto på lästa.
- **Moderation:** Helt ny moderation.js med blockering och rapportering, plus tabellerna user_blocks och reports.
- **Användarskapade communities:** POST /api/communities med community-namer.js för AI-förslag på namn, plus creator_id-kolumnen och creator-spårning.
- **Säkerhet på auth:** attempts-räknare med invalidering vid 5 felaktiga försök, crypto.timingSafeEqual, sessionsrotation (max 10 per användare, rensning vid >90 dagar).
- **GDPR:** GET /api/users/me/export ger nedladdningsbar JSON med all data.
- **Bilder:** Magic-byte-validering av avатарer, inte bara Content-Type.
- **Privacy:** /api/users/:id kräver att man delar community med målet — IDOR-skydd.
- **ML-cache:** Modellen serialiseras till data/model.json och laddas om datasetet inte ändrats.
- **Rate-limiting:** Egen rate-limit.js används brett (meddelanden, inlägg, AI-preview, rapporter), inte bara SMS-koder.
- **PII-varning** i meddelandeklienten.

8. Vad jag skulle ifrågasätta (kort)

Eftersom du bad om en mentorsblick, inte bara en sammanfattning:

- **writeFileSync på avатарer i request-tråden** blockerar event-loopen. Litet i praktiken (5 MB max), men det är det enda stället backend faktiskt blockerar — fs.promises.writeFile är en gratis fix.
- **In-memory rate-limit** dör vid omstart och funkade inte om du någonsin kör flera processer. Det är en medveten kompromiss för en single-process-demo, men värt att markera som tekniskt skuld om sajten skulle skalas.
- **SSE-anslutningar saknar tak per användare.** En mottagare kan i princip ha tusentals öppna flikar, alla skriver vid varje meddelande. Lågt riskvärde, men trivalt att lägga ett tak på 5-10 strömmar per userId för symmetri med session-taket.
- **auth_codes rensas bara per nummer vid lyckad verify eller när max attempts nås.** Ingen periodisk garbage collection — utgångna koder kan ligga kvar i månader. En DELETE FROM auth_codes WHERE expires_at < ? vid uppstart (i stil med sessionsrensningen) skulle vara konsekvent.
- **community-namer är deterministisk per intresse-set + stad.** Bra för testning, dåligt för UX: två användare i Stockholm med samma 3 intressen får exakt samma föreslagna namn. Lägg till userId eller en tidskomponent i hash-seedet om det visar sig på riktigt.
- **getCurrentUser() cachar i en modulvariabel utan tidsstämpel.** Om en annan flik uppdaterar profilen ser den här fliken den gamla versionen tills sidan laddas om. Inget akut, men något att veta.
- **Den gamla rapporten ligger kvar i två exemplar** (teknisk-rapport.md i roten och docs/teknisk-rapport.md). Antingen flytta båda till docs/ och uppdatera, eller radera dubletten.

Inget av det här är blockerande — det är en välstrukturerad prototyp där genomtänkta säkerhets- och tillgänglighetsval syns på rätt ställen. Men du bad om en ärlig läsning, så där är den.